- STM32 平衡车程序代码详解教程（超详细版）
  - 目录
- 第 0 章 工程全景
  - 0.1 这套程序要解决什么问题
  - 0.2 硬件资源与引脚分配
  - 0.3 时钟基础：72MHz 从哪来
  - 0.4 工程目录结构
- 第 1 章 程序骨架：main.c 逐行精讲
  - 1.1 头文件包含
  - 1.2 全局变量逐个讲清
  - 1.3 三个 PID 控制器的出厂参数
  - 1.4 初始化顺序大有讲究
  - 1.5 主循环 while(1) 逐段精讲
  - 1.6 中断调度中心 TIM1_UP_IRQHandler 逐行精讲
- 第 2 章 PID 控制器：PID.c / PID.h
  - 2.1 数据结构：一个 PID_t 描述一切
  - 2.2 从连续公式到离散代码
  - 2.3 PID_Init：状态清零的学问
  - 2.4 PID_Update 逐行精讲
- 第 3 章 姿态感知：MyI2C / MPU6050 / 互补滤波
  - 3.1 为什么必须两种传感器互补
  - 3.2 软件 I2C 的地基：开漏输出
  - 3.3 六个协议函数：把 I2C 时序图”翻译”成代码
  - 3.4 MPU6050 寄存器地图
  - 3.5 MPU6050.c 逐函数精讲
  - 3.6 互补滤波系数 0.01 的推导
- 第 4 章 执行机构：PWM / Motor / Encoder
  - 4.1 PWM.c：20kHz 调速的诞生过程
  - 4.2 Motor.c：有符号速度 → H 桥方向 + 占空比

    - 4.3 Encoder.c：正交编码与四倍频
- 第 5 章 系统节拍：Timer / Delay / 中断系统
    - 5.1 Timer.c：1kHz 心跳
    - 5.2 Delay.c：不占用定时器外设的 SysTick 延时
- 第 6 章 人机交互：Key / LED / OLED
    - 6.1 Key.c：中断驱动的状态机消抖
    - 6.2 LED.c：三行代码 + 灌电流常识
    - 6.3 OLED.c：一款完整显示驱动的解剖
- 第 7 章 串行通信：Serial / BlueSerial
    - 7.1 初始化配置逐项解读
    - 7.2 发送：从 SendByte 到 printf
    - 7.3 BlueSerial 的包接收状态机（重点）
    - 7.4 主循环侧的协议落地
- 第 8 章 工程组织与其它文件
    - 8.1 stm32f10x_it.c：空的中断模板
    - 8.2 头文件的组织规则
    - 8.3 stm32f10x_conf.h：标准库的总闸
- 第 9 章 调参方法与常见问题 FAQ
    - 9.1 调参顺序（血泪总结）
    - 9.2 症状速查表
    - 9.3 实时性账本（中断预算 1ms）
- 第 10 章 思考题与进阶方向
    - 10.1 各章思考题汇总
    - 10.2 进阶改造路线
- 附录 A 引脚分配总表
- 附录 B 全局符号速查表

STM32 平衡车程序代码详解教程（超详细版）

> 基于《江协科技 平衡车入门教程》“06-平衡车程序优化”工程源码 主控：STM32F103C8T6（72MHz Cortex-M3）· 标准外设库 · 串级 PID 控制 本文逐文件、逐函数、逐行讲解全部自编代码，包含原理推导、寄存器细节、时序图、调参方法与常见问题。

目录

- 第 0 章 工程全景
- 第 1 章 程序骨架：main.c 逐行精讲
- 第 2 章 PID 控制器：PID.c / PID.h
- 第 3 章 姿态感知：MyI2C / MPU6050 / 互补滤波
- 第 4 章 执行机构：PWM / Motor / Encoder
- 第 5 章 系统节拍：Timer / Delay / 中断系统
- 第 6 章 人机交互：Key / LED / OLED
- 第 7 章 串行通信：Serial / BlueSerial
- 第 8 章 工程组织与其它文件
- 第 9 章 调参方法与常见问题 FAQ
- 第 10 章 思考题与进阶方向
- 附录 A 引脚分配总表
- 附录 B 全局符号速查表

第 0 章 工程全景

0.1 这套程序要解决什么问题

两轮自平衡车的本质是一个**倒立摆**：重心在轮轴上方，天然不稳定，必须实时测量车身倾角、并驱动车轮向”倒下的方向”运动，把车底”接”回重心下方。整个过程要求：

1. **感知**：MPU6050 测出车身倾角（加速度计 + 陀螺仪融合）；
2. **决策**：串级 PID 计算需要的电机输出（直立环 + 速度环 + 转向环）；
3. **执行**：TIM2 输出 PWM，经驱动芯片控制两个带编码器的减速电机；
4. **节拍**：TIM1 以 1kHz 中断作为整个系统的”心跳”，保证控制周期严格稳定。

```
flowchart LR
    R[蓝牙摇杆<br>目标速度/转向] --> B
    A["MPU6050<br>姿态角 Angle"] --> C
    B["速度环 SpeedPID<br>50ms"] -- "输出→目标角度" --> C
    C["直立环 AnglePID<br>10ms"] -- "AvePWM" --> E["合成+限幅<br>Left/RightPWM"]
    T["转向环 TurnPID<br>50ms"] -- "DifPWM" --> E
    E --> M["电机×2<br>带编码器"]
    M -. "编码器反馈" .-> B
    M -. "编码器反馈" .-> T
```

为什么是”串级”？ 速度环的输出不是 PWM，而是**直立环的目标角度**。想让车前进，就先让它”往前倾”——倾角产生加速度，速度才会变化。外环（速度）慢（50ms）、内环（直立）快（10ms），外环的输出作为内环的设定值，这是工业控制中典型的串级结构。

0.2 硬件资源与引脚分配

外设	引脚	模式	作用
TIM2 CH1/CH2	PA0 / PA1	复用推挽	20kHz 电机 PWM
GPIOB	PB12~PB15	推挽输出	左右电机方向控制（IN1/IN2）
TIM3 CH1/CH2	PA6 / PA7	上拉输入	左电机编码器 A/B 相
TIM4 CH1/CH2	PB6 / PB7	上拉输入	右电机编码器 A/B 相
软件 I2C	PB10 / PB11	开漏输出	MPU6050（SCL/SDA）
软件 I2C	PB8 / PB9	开漏输出	OLED（SCL/SDA）
USART1	PA9 / PA10	复用/上拉	调试串口（9600-8-N-1）
USART2	PA2 / PA3	复用/上拉	蓝牙模块（9600-8-N-1）
GPIOB/GPIOA	PB1/PB0/PA5/PA4	上拉输入	4 个用户按键（低电平有效）
GPIOC	PC13	推挽输出	板载 LED（低电平点亮）

注意一个细节：**OLED 和 MPU6050 各用一套独立软件 I2C**，互不复用。这样两套设备的时序完全独立，不需要处理 I2C 多从机寻址切换的复杂性——用引脚换来了简单。

0.3 时钟基础：72MHz 从哪来

STM32F103 上电默认用内部 8MHz HSI。本工程依赖启动代码 `system_stm32f10x.c` 中的 `SystemInit()` 完成时钟配置：

```
HSE 外部晶振 8MHz ──×9(PLL)──▶ SYSCLK 72MHz ──▶ AHB 72MHz
                                                  ├──▶ APB1 分频2 = 36MHz
（TIM2/3/4 挂这里）
                                                  └──▶ APB2 不分频 = 72MHz（TIM1/
GPIO/USART1/ADC）
```

关键规则：挂在 APB1（36MHz）上的定时器，如果 APB1 分频系数不为 1，则定时器时钟自动 ×2，所以 **TIM2/TIM3/TIM4 的计数时钟同样是 72MHz**。后面所有 PSC 计算都以 72MHz 为基准。

0.4 工程目录结构

```
06-平衡车程序优化
├── Start     启动文件 stm32f10x.s（中断向量表）、system_stm32f10x.c（时钟）、core_cm3.c（内核）
├── Library   ST 标准外设库（stm32f10x_gpio/tim/usart/rcc... + misc.c）
├── User      main.c（主逻辑+中断）、PID.c/h、stm32f10x_it.c、stm32f10x_conf.h
├── System    Timer.c/h（TIM1 节拍）、Delay.c/h（SysTick 延时）
└── Hardware  Motor/PWM/Encoder/MPU6050/MyI2C/OLED/OLED_Data/Key/LED/Serial/BlueSerial
```

分层思想：`Start → Library → System → Hardware → User`，上层只调用下层接口。所有”与具体硬件接线有关”的代码被压到各模块的引脚配置函数里，换引脚只改一处。

第 1 章 程序骨架：main.c 逐行精讲

main.c 是全车的大脑：全局数据、PID 参数、初始化流程、主循环（显示 + 遥控解析）、以及 1kHz 中断调度中心都在这里。

1.1 头文件包含

```
#include "stm32f10x.h"    // 汇总了器件寄存器定义 + stm32f10x_conf.h（标准库全部头文件）
#include "Delay.h"        // SysTick 延时（本工程 main 中未直接用，模块内部用）

#include "OLED.h"         // OLED 驱动接口
#include "LED.h" "Timer.h" "Key.h" "MPU6050.h" "Motor.h" "Encoder.h"
#include "Serial.h"       // USART1 调试串口
#include "BlueSerial.h"   // USART2 蓝牙
#include "PID.h"          // PID 结构体与算法
#include <math.h>         // atan2（姿态解算）
#include <string.h>       // strcmp/strtok（协议解析）
#include <stdlib.h>       // atof/atoi（字符串转数值）
```

`stm32f10x.h` 顶部有 `#include "stm32f10x_conf.h"`（需定义 `USE_STDPERIPH_DRIVER`），后者把所有标准库 .h 一网打尽，所以业务代码只写一句器件头即可。

1.2 全局变量逐个讲清

```
int16_t AX, AY, AZ, GX, GY, GZ;
```

MPU6050 六轴原始值。`int16_t` 是**有符号** 16 位，对应传感器输出 `(高字节<<8)|低字节` 的补码格式。以陀螺仪 ±2000°/s 量程为例，灵敏度 16.4 LSB/(°/s)：读到 +32768 表示 +2000°/s。全车只用到了 `AX、AZ（算倾角）` 和 `GY（俯仰角速度）`，其余留作扩展。

```
uint8_t TimerErrorFlag;   // 中断超时报警标志：=1 表示某次中断执行时间超过 1ms

uint16_t TimerCount;      // 中断退出时 TIM1 计数器快照，用于估算中断耗时
```

这两个是**调试设施**：中断函数末尾会二次检查更新标志（见 1.6），若上次中断没处理完下一次更新又来了，说明 10ms/50ms 分支太耗时，`TimerErrorFlag` 置 1，可在调试器/串口观察。

```
float AngleAcc;    // 加速度计单独解算出的角度（噪声大、不漂移）
float AngleGyro;   // 陀螺仪积分得到的角度（平滑、会漂移）
float Angle;       // 二者互补融合后的最终角度 —— 全车的核心状态量
```

```
uint8_t KeyNum, RunFlag;
```

`KeyNum` 暂存按键号（1~4，0 表示无）；`RunFlag` 平衡车运行开关：1=电机受控，0=电机停转。

```
int16_t LeftPWM, RightPWM;   // 送入 Motor_SetPWM 的最终值，范围 ±100
int16_t AvePWM, DifPWM;      // 平均分量（直立环输出）与差动分量（转向环输出）
```

PWM 用**有符号数**表示”大小 + 方向”，正负号在 `Motor_SetPWM` 内部翻译成方向引脚电平。合成公式（中断里）：

```
LeftPWM  = AvePWM + DifPWM/2      （左轮 = 前进分量 + 转向分量）
RightPWM = AvePWM - DifPWM/2      （右轮 = 前进分量 - 转向分量）
```

```
float LeftSpeed, RightSpeed;   // 左右轮实测速度（单位见 1.6 换算说明）
float AveSpeed, DifSpeed;      // 平均速度（速度环反馈）、速度差（转向环反馈）
```

1.3 三个 PID 控制器的出厂参数

```
PID_t AnglePID = {
    .Kp = 5,     .Ki = 0.1,   .Kd = 5,
    .OutMax = 100,  .OutMin = -100,     // 输出限幅 = PWM 满量程
    .OutOffset = 3,                     // 死区补偿
    .ErrorIntMax = 600, .ErrorIntMin = -600,
};
```

参数	直立环 AnglePID	速度环 SpeedPID	转向环 TurnPID	含义
Kp	5	2	4	误差 → 输出的比例强度
Ki	0.1	0.05	3	消除稳态误差
Kd	5	0	0	直立环用”微分先行”，其余未用
Out 限幅	±100	±20	±50	速度环输出是”目标角度”（°），所以只给 ±20
积分限幅	±600	±150	±20	积分项最大贡献 = Ki×600 = 60 PWM

速度环 Kd=0、转向环 Kd=0 不是偷懒：外环周期 50ms 且反馈本身经过机械滤波，微分项只会放大噪声；而直立环 10ms 快速响应中 D 项（阻尼）必不可少。

`OutMax=±20` 的物理含义：速度环最多命令直立环”目标角 ±20°“，配合主循环的 `|Angle|>50°` 保护，形成两级安全边界。

1.4 初始化顺序大有讲究

```
OLED_Init();         // 1. 显示最先好，方便后续调试观察
MPU6050_Init();      // 2. 传感器就绪（内部含 MyI2C_Init + 6 条配置命令）
BlueSerial_Init();   // 3. 蓝牙：越早开越好，随时可能收到调参指令
LED_Init(); Key_Init(); Motor_Init(); Encoder_Init(); Serial_Init();

Timer_Init();        // 4. 最后才开 1kHz 中断！
```

为什么 Timer_Init 必须放最后？ `Timer_Init` 一执行，1kHz 中断立刻开始跑，中断里会读 MPU6050、读编码器、调用 PID——如果此时某个外设还没初始化，中断将读到垃圾数据甚至卡死（例如 I2C 未初始化时读 MPU6050）。“外设先就位、心跳最后启动”是实时系统的通用原则。

1.5 主循环 while(1) 逐段精讲

主循环承担三件事：按键启停、OLED 刷新、蓝牙协议解析。它跑得越快越好，控制本身不在这里。

（1）运行指示与按键启停

```
        if (RunFlag) {LED_ON();} else {LED_OFF();}    // LED 常亮 = 运行中
        KeyNum = Key_GetNum();                         // 取键值（读后自动清零）
        if (KeyNum == 1)
        {
            if (RunFlag == 0)
            {
                PID_Init(&AnglePID);   // 清 Target/Actual/ErrorInt/Out 等全部历史状态

                PID_Init(&SpeedPID);
                PID_Init(&TurnPID);
                RunFlag = 1;
            }
            else { RunFlag = 0; }
        }
```

启动前必须 `PID_Init` 三个控制器：车停过一段时间，积分器里可能残留上次运行的误差累积；不清理的话，一启动积分项就会输出一个”莫名的偏置”，车会窜一下。清零后一切从头积累。

（2）OLED 三列调试界面

```
        OLED_Clear();                    // 只清显存数组（内存操作，很快）
        OLED_Printf(0,  0, OLED_6X8, "  Angle");    // 第1列：直立环
        OLED_Printf(0,  8, OLED_6X8, "P:%05.2f", AnglePID.Kp);
        ...
        OLED_Printf(0, 48, OLED_6X8, "O:%+05.0f", AnglePID.Out);   // 输出
        OLED_Printf(0, 56, OLED_6X8, "GY:%+05d", GY);              // 原始陀螺值（看零偏）
        OLED_Printf(56,56, OLED_6X8, "Offset:%02.0f", AnglePID.OutOffset);
        OLED_Printf(50, 0, ...  "Speed");            // 第2列：速度环（50像素起）
        OLED_Printf(88, 0, ...  "Turn");             // 第3列：转向环（88像素起）
        OLED_Update();                   // 1KB 显存经软 I2C 整屏发出
```

`%05.2f` 表示宽度 5、不足补零、两位小数；`%+05.1f` 的 `+` 强制显示正负号，数值变化时符号位不跳动，读数更稳。这一整块界面就是**在线调参仪表盘**：手机改 PID → 屏幕立即确认写入值，屏幕同时显示 Target/Actual/Out 三要素，调参时一眼看出”误差多大、输出了多少”。

> ⚠ 性能账本：`OLED_Update()` 要通过软件 I2C 发送 8页×128字节 = 1024 字节 ≈ 9216 个 SCL 时钟+额外时序开销。按 GPIO 翻转约 0.5~1MHz 的位速率估算，一次全屏刷新要 **十几毫秒**。所以主循环实际是”几 ms 一圈”，而不是”越快越好”。好在控制都放在中断里，主循环慢一点只影响显示/响应流畅度，不影响控制稳定性——这正是中断调度的价值。

（3）蓝牙协议解析（strtok 三连）

```
        if (BlueSerial_RxFlag == 1)      // 中断收完一包 [xxx] 后置 1
        {
            char *Tag = strtok(BlueSerial_RxPacket, ",");   // 按逗号切第一段
```

`strtok` 是 C 标准库函数：第一次调用传入字符串，后续传 `NULL` 继续切同一字符串。它会把分隔符替换成 `\0` 并返回段指针。协议三种包：

① slider 滑块（在线调参），格式 `[slider,AngleKp,5.5]`：

```
        char *Name  = strtok(NULL, ",");
        char *Value = strtok(NULL, ",");
        if (strcmp(Name, "AngleKp") == 0)  AnglePID.Kp  = atof(Value);
        else if (strcmp(Name, "SpeedKi") == 0) SpeedPID.Ki = atof(Value);
        ...   // 共覆盖 9 个 PID 参数 + Offset（OutOffset 死区补偿）
```

`atof` 把 `"5.5"` 变成 `5.5f`。手机 App 上拖滑块 → 车上参数实时生效 → 看车况 → 再拖，这就是”在线整定”的完整闭环。

② joystick 摇杆（遥控），格式 `[joystick,0,50,0,0]`：

```
        int8_t LH = atoi(strtok(NULL, ","));   // 左摇杆水平 -128~127
        int8_t LV = atoi(strtok(NULL, ","));   // 左摇杆垂直
        int8_t RH = atoi(strtok(NULL, ","));   // 右摇杆水平
        int8_t RV = atoi(strtok(NULL, ","));
        SpeedPID.Target = LV / 25.0;    // 推杆最大约 ±127 → 目标速度约 ±5
        TurnPID.Target  = RH / 25.0;    // 右摇杆控制转向目标
```

`/25.0` 把摇杆刻度换算成控制器量纲。注意 `int8_t`：App 若发来 0~255 无符号值，超过 127 会变负数，遥控”半程反跳”就是这个原因。

③ **key 按键包**：解析了 Name/Action 但函数体为空，属于**预留接口**（App 按钮事件）。

最后 `BlueSerial_RxFlag = 0;` 允许接收下一包。

> ⚠ 数据竞争提示：中断随时可能写 `BlueSerial_RxPacket`。这里靠 `RxFlag==1 期间中断丢弃新包`（见第 7 章）保护，主循环解析期间新来的完整包会被丢掉——对遥控场景可接受，但这是”丢包保一致”的取舍，不是严格互斥。

1.6 中断调度中心 TIM1_UP_IRQHandler 逐行精讲

这是全车**实时性核心**，请对照源码逐行看：

```
        void TIM1_UP_IRQHandler(void)
        {
            static uint16_t Count0, Count1;      // static：跨调用保持。两个软件分频计数器
```

`static` 局部变量存放在全局区而非栈上，函数退出值不丢。`Count0` 分频出 10ms（姿态+直立环），`Count1` 分频出 50ms（速度+转向环）。

```
            if (TIM_GetITStatus(TIM1, TIM_IT_Update) == SET)
            {
                TIM_ClearITPendingBit(TIM1, TIM_IT_Update);   // 尽早清标志，为下一次中断做准备
                Key_Tick();                                    // 每 1ms 喂一次按键状态机
```

（1）10ms 分支：姿态解算 + 直立环（100Hz）

```
            Count0 ++;
            if (Count0 >= 10)      // 10 × 1ms = 10ms
            {
                Count0 = 0;
                MPU6050_GetData(&AX, &AY, &AZ, &GX, &GY, &GZ);    // 突发读 14 字节

                GY -= 16;     // 陀螺零偏扣除（见下）
```

`GY -= 16` 的来历：±2000°/s 量程下灵敏度 16.4 LSB/(°/s)，静止时实测 GY≈+16，即约 1°/s 的零偏。若不扣，1°/s 积分到互补滤波里，10 秒就漂 10°，车必倒。这个 16 是**上电时车静止不动时实测标定**的——换传感器必须重标。

```
                AngleAcc = -atan2(AX, AZ) / 3.14159 * 180;    // 加速度计解算角度

                AngleAcc += 0.5;                              // 机械零点修正
```

`atan2(AX, AZ)` 返回重力方向与 Z 轴的夹角（弧度，±π）。车身前倾 θ 时，重力在 X 轴分量增大、Z 轴减小，`atan2(AX/AZ)` 即 θ。前面的负号来自传感器**贴装方向**（X 轴正方向与”前倾”相反）；`+0.5` 是**机械安装零点**——重心/传感器不可能绝对水平，实测静止时显示 0.5°，就直接加上去抵消。

```
                AngleGyro = Angle + GY / 32768.0 * 2000 * 0.01;    // 陀螺仪积分角度
```

逐项拆解：- `GY / 32768.0 * 2000`：原始值 → 角速度（°/s）。32768 是 ±2000°/s 满量程对应的码值；- `* 0.01`：°/s × 0.01s（本周期时长 10ms）= **这 10ms 里转过的角度增量**；- `Angle + ...`：上一周期的融合角 + 本周期增量 = 陀螺仪口径的当前角。**积分就在这一个 `+` 号里**。

```
                float Alpha = 0.01;
                Angle = Alpha * AngleAcc + (1 - Alpha) * AngleGyro;    // 互补滤波
```

一行融合两种传感器：陀螺积分（高通特性，短期准）权重 0.99，加速度计角度（低通特性，长期准）权重 0.01。等效于对陀螺做一阶高通、对加计做一阶低通再相加，时间常数 τ ≈ T×(1−α)/α ≈ 0.99s——**秒级内信陀螺，分钟级信加计**，加计的抖动被压到几乎不可见，陀螺的漂移被持续拉回。

```
                    if (Angle > 50 || Angle < -50) { RunFlag = 0; }   // 倒地保护
```

倾角超过 ±50° 说明车已经倒了，强制停机，避免轮子疯转、电机堵转过流。重启需按键重新启动（并重置 PID）。

```
                    if (RunFlag)
                    {
                        AnglePID.Actual = Angle;       // 反馈量：当前角度
                        PID_Update(&AnglePID);         // 计算直立环输出
                        AvePWM = -AnglePID.Out;        // ← 这个负号是"符号约定"的枢纽
```

`AvePWM = -AnglePID.Out` 的负号：直立环的正向误差 = “目标角 − 实际角”。假设目标 0°、实际后仰 +5°，希望电机**向后开**把车底接到重心下。而 `AnglePID.Out = Kp×(0−5) = −25`（负值），若直接送电机则车轮向前——方向反了。工程上不改传感器方向定义，就在输出处统一取负补偿。**符号问题是平衡车第一大坑**，任何一处装反（传感器方向/电机接线/编码器方向）都表现为”一启动就朝一个方向加速到倒”。

```
                        LeftPWM  = AvePWM + DifPWM / 2;    // 差速合成
                        RightPWM = AvePWM - DifPWM / 2;
                        if (LeftPWM  > 100) {LeftPWM  = 100;} else if (LeftPWM  < -10
0) {LeftPWM  = -100;}
                        if (RightPWM > 100) {RightPWM = 100;} else if (RightPWM < -10
0) {RightPWM = -100;}
                        Motor_SetPWM(1, LeftPWM);
                        Motor_SetPWM(2, RightPWM);
                    }
                    else { Motor_SetPWM(1, 0); Motor_SetPWM(2, 0); }   // 停机时两轮刹车
```

差速合成是”加法叠加”：转向不再单独占一路 PWM，而是把左右轮”一加一减”。限幅 ±100 与 PWM 分辨率（ARR=99）一致，防溢出到方向逻辑之外。

（2）50ms 分支：测速 + 速度环/转向环（20Hz）

```
                Count1 ++;
                if (Count1 >= 50)     // 50 × 1ms = 50ms
                {
                    Count1 = 0;
                    LeftSpeed  = Encoder_Get(1) / 44.0 / 0.05 / 9.27666;
                    RightSpeed = Encoder_Get(2) / 44.0 / 0.05 / 9.27666;
```

换算链逐项拆：- `Encoder_Get(n)`：50ms 内编码器计数的**增量**（含符号，四倍频计数）；- `/ 44.0`：输出轴转一圈的计数次数 → 得到”转了百分之几圈”；- `/ 0.05`：除以采样周期（秒）→ 得到 **圈/秒**；- `/ 9.27666`：最终标定系数，把”圈/秒”折算成控制器用的统一速度单位（与轮径、减速比相关的实测值）。

44 和 9.27666 都是标定值：让车架空、手转轮子一圈，读屏上的计数即可验证 44；9.27666 只要左右一致、标度统一即可（它会被速度环 Kp 吸收），**关键在于左右轮用同一系数**，否则走直线会跑偏。

```
                    AveSpeed = (LeftSpeed + RightSpeed) / 2.0;   // 平均速度 → 速度环反馈
                    DifSpeed = LeftSpeed - RightSpeed;           // 速度差   → 转向环反馈

                    if (RunFlag)
                    {
                        SpeedPID.Actual = AveSpeed;
                        PID_Update(&SpeedPID);
                        AnglePID.Target = SpeedPID.Out;      // ⊢ 串级枢纽：外环输出 = 内环目标

                        TurnPID.Actual = DifSpeed;
                        PID_Update(&TurnPID);
                        DifPWM = TurnPID.Out;                // 转向环输出直接是差动 PWM

                    }
                }
```

`AnglePID.Target = SpeedPID.Out` 是全车最核心的一行：速度偏快 → 外环输出负角度目标 → 直立环让车**主动后仰** → 减速。理解了这行，就理解了平衡车速度控制的全部秘密。转向环同理：给右摇杆目标 → 与 `DifSpeed` 比较 → 输出 DifPWM 让两轮速度不同。

（3）尾部超时自检（精妙）

```
                if (TIM_GetITStatus(TIM1, TIM_IT_Update) == SET)   // 处理完了标志又是
SET？
                {
                    TimerErrorFlag = 1;                            // 说明本次执行 >
1ms，超时
                    TIM_ClearITPendingBit(TIM1, TIM_IT_Update);
                }
                TimerCount = TIM_GetCounter(TIM1);   // 快照当前计数（0~999），可估算耗时

            }
        }
```

原理：若中断体在 1ms 内跑完，更新标志已清，不会再次 SET；若**超过 1ms**，TIM1 已再次溢出、标志再次置位——在这里被逮个正着。`TimerCount` 读的是 ARR=999 的递增计数器当前值，配合断点可换算中断执行到此刻已经过去多少 μs。这是无示波器条件下检验实时性的土办法，非常值得学习。

中断调度全景图：

```
flowchart TD
    T["TIM1 更新中断 · 每 1ms"] --> K["Key_Tick 按键扫描"]
    K --> C0{"Count0 ≥ 10 ?"}
    C0 -- "是 → 10ms 分支" --> S1["读 MPU6050 → 扣零偏<br>atan2 解算 + 互补滤波
<br>±50° 倒地保护"]
    S1 --> RF{"RunFlag ?"}
    RF -- "运行" --> S2["直立环 PID_Update<br>差速合成 + 限幅<br>Motor_SetPWM
×2"]
    RF -- "停止" --> S3["Motor_SetPWM(…, 0)"]
    C0 -- "否" --> C1{"Count1 ≥ 50 ?"}
    S2 --> C1
    S3 --> C1
    C1 -- "是 → 50ms 分支" --> S4["Encoder_Get 读清零<br>速度换算 AveSpeed/
DifSpeed<br>速度环输出→AnglePID.Target<br>转向环输出→DifPWM"]
    C1 -- "否" --> D["尾部超时自检<br>TimerErrorFlag / TimerCount"]
    S4 --> D
```

本章思考： 1. 若把 `Timer_Init()` 挪到 `MPU6050_Init()` 之前，上电会发生什么？ 2. `AngleAcc += 0.5` 里 0.5 换一辆车装配后还适用吗？怎么重新测定？ 3. 为什么停止时

`Motor_SetPWM(1, 0)` 要显式调用，而不是”不调用就不动”？（提示：上次中断留下的 PWM 是多少？）

第 2 章 PID 控制器：PID.c / PID.h

2.1 数据结构：一个 PID_t 描述一切

PID.h 用一个结构体把控制器全部状态打包：

```
typedef struct {
    float Target;       // 目标值（设定值 SP）
    float Actual;       // 反馈值（测量值 PV），每次调用前由外部刷新
    float Actual1;      // 上一次的反馈值 —— 微分先行专用
    float Out;          // 输出

    float Kp, Ki, Kd;   // 三个增益

    float Error0;       // 本次误差  = Target - Actual
    float Error1;       // 上次误差（本版本未被微分项使用，保留）
    float ErrorInt;     // 误差积分器（累加和）

    float ErrorIntMax, ErrorIntMin;   // 积分限幅（抗饱和）
    float OutMax, OutMin;             // 输出限幅
    float OutOffset;                  // 死区补偿量
} PID_t;
```

为什么用结构体 + 指针而不是全局变量？ 三个控制器（Angle/Speed/Turn）字段完全同构，`PID_Update(&AnglePID)` 传指针即可复用同一份算法代码——这就是”数据与算法分离”。

2.2 从连续公式到离散代码

连续 PID：

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$$

采样周期 T 内离散化（本工程直立环 T=10ms、速度/转向环 T=50ms）：

- 积分 → 累加：$\int e\,dt \approx T\Sigma e_k$，工程上把 T 折进 Ki；
- 微分 → 差分：$\frac{de}{dt} \approx \frac{e_k - e_{k-1}}{T}$，T 折进 Kd。

于是代码中的输出：

$$Out_k = K_p e_k + K_i \Sigma_i e_i - K_d (Actual_k - Actual_{k-1})$$

2.3 PID_Init：状态清零的学问

```
void PID_Init(PID_t *p)
{
    p->Target = 0;
    p->Actual = 0;
    p->Actual1 = 0;     // 不清它，第一次微分 = Actual - 垃圾值，输出猛跳一下

    p->Out = 0;
    p->Error0 = 0;
    p->Error1 = 0;
    p->ErrorInt = 0;    // 积分器清零：上次运行的累积必须作废
}
```

注意它**不清 Kp/Ki/Kd/限幅**——那些是”参数”，Init 只清”状态”。启动平衡车前对三个控制器各调一次（见 1.5），避免旧积分残留导致起步窜动。

2.4 PID_Update 逐行精讲

```
void PID_Update(PID_t *p)
{
    p->Error1 = p->Error0;              // 旧误差存档
    p->Error0 = p->Target - p->Actual; // 计算新误差
```

（1）积分项与抗饱和（Anti-Windup）

```
            if (p->Ki != 0)
            {
                p->ErrorInt += p->Error0;
                if (p->ErrorInt > p->ErrorIntMax) {p->ErrorInt = p->ErrorIntMax;}

                if (p->ErrorInt < p->ErrorIntMin) {p->ErrorInt = p->ErrorIntMin;}

            }
            else { p->ErrorInt = 0; }
```

- **积分饱和**是什么：误差长时间同号（如车被手按住），ErrorInt 一路涨。即使误差反向了，巨大的积分累积还要慢慢”泄洪”，输出迟迟回不来——车表现为”抽风几秒才稳”。
- **限幅即抗饱和**：给 ErrorInt 设上限，超出就钳住。直立环 ±600 × Ki 0.1 = 积分项最大贡献 ±60 PWM，既有消除稳态倾斜的力量，又不会失控。
- `Ki==0` 时干脆清零积分器：速度环若临时把 Ki 调成 0 在线测试，积分器自动归零，避免”看起来关了其实还在输出”的迷惑。

（2）微分先行：`-Kd*(Actual - Actual1)` 的深意

```
            p->Out = p->Kp * p->Error0
                   + p->Ki * p->ErrorInt
        //         + p->Kd * (p->Error0 - p->Error1);     // 标准（被注释）：对误差微分

                   - p->Kd * (p->Actual - p->Actual1);     // 实际采用：对测量值微分
```

标准差分 $\frac{e_k-e_{k-1}}{T}$ 在**目标值突变**时会产生巨大的微分尖峰（derivative kick）：平衡车的 AnglePID.Target 每个速度环周期都被改写（50ms 一次阶跃），若对误差微分，每次阶跃 D 项都轰一下输出，车会”一冲一冲”。

微分先行把微分对象换成**测量值 Actual**：目标平滑变化时不尖峰；车身姿态突变（真正需要阻尼的时刻）时，Actual 的变化照样产生有效阻尼。注意前面的**负号**：误差 = Target − Actual，对 Actual 微分等价于对误差微分取负号（Target 变化不参与），公式上 `-Kd·ΔActual = Kd·(e_k − e_{k-1}) − Kd·ΔTarget`，正好消去了 ΔTarget 项。

（3）死区补偿 OutOffset

```
        if (p->Out > 0) {p->Out += p->OutOffset;}
        if (p->Out < 0) {p->Out -= p->OutOffset;}
```

电机有**死区**：PWM 太小（要克服静摩擦 + 驱动管压降）轮子不动。表现为小信号时”输出 10 和输出 0 效果一样”。补偿方法：只要输出非零，就额外加（正向）或减（反向）一点，把输出”垫”过死区。`Out==0` 时不补偿——静止就是静止，不加偏置。

> 注意 `OutOffset` 只对直立环有意义（默认 3），对速度环/转向环（输出是目标值而非 PWM）应为 0。

（4）输出限幅与状态存档

```
        if (p->Out > p->OutMax) {p->Out = p->OutMax;}
        if (p->Out < p->OutMin) {p->Out = p->OutMin;}

        p->Actual1 = p->Actual;     // 保存本次反馈，供下次微分用
    }
```

限幅放在**所有计算之后**：内部 ErrorInt 仍是真实值，只是输出被钳住——这样退饱和时积分器状态是连续的。

一次完整调用的时序：外部填 `p->Actual` → Update 算出 `p->Out` → 外部取走 Out。.Target 由更上层（速度环输出、遥控）写入。

第 3 章 姿态感知：MyI2C / MPU6050 / 互补滤波

3.1 为什么必须两种传感器互补

	加速度计 → AngleAcc	陀螺仪 → AngleGyro
原理	测重力在各轴的分量，atan2 反解倾角	测角速度，累加（积分）成角度
静止	很准	漂移（零偏被积分放大）
运动	混入车体加速度，瞬间大偏差	不受线加速度影响，平滑
频域特性	低频准、高频噪声大 → **低通**	高频准、低频漂移 → **高通**

单独用任何一个都立不起来。互补滤波 `Angle = α·AngleAcc + (1−α)·AngleGyro` 恰好让两者各取所长（详见 3.6）。

3.2 软件 I2C 的地基：开漏输出

MyI2C.c 使用 PB10(SCL)/PB11(SDA)，配置为**开漏输出**（GPIO_Mode_Out_OD）+ 外部上拉：

- 开漏模式只能主动输出低电平，输出”1”时引脚悬空、靠上拉电阻拉高；
- 这实现了 I2C 的**线与**逻辑：任何一个器件拉低，总线就是低——这是 ACK、时钟拉伸、多主机仲裁的基础；
- 读 SDA 前必须先写 1（释放总线），否则自己钳着低电平永远读到 0。

```
        void MyI2C_W_SCL(uint8_t BitValue) { GPIO_WriteBit(GPIOB, GPIO_Pin_10, (BitAc
tion)BitValue); }
        void MyI2C_W_SDA(uint8_t BitValue) { GPIO_WriteBit(GPIOB, GPIO_Pin_11, (BitAc
tion)BitValue); }
        uint8_t MyI2C_R_SDA(void)          { return GPIO_ReadInputDataBit(GPIOB,
GPIO_Pin_11); }
```

`(BitAction)BitValue` 把 0/1 强转成标准库枚举。注释掉的 `Delay_us(10)`：72MHz 的 M3 翻转 GPIO 约 100ns 级，远快于 MPU6050 承受的 400kHz 上限其实也够慢（软 I2C 每字节 9 个循环

+函数调用开销，实测约 100~200kHz），72MHz 下无需延时。**若换成 180MHz 的 F4/H7 就必须打开延时**，这正是注释留着的意义。

初始化：

```
        void MyI2C_Init(void)
        {
            RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);    // 先开时钟！忘了这行=寄存器全无效
            GPIO_InitTypeDef GPIO_InitStructure;
            GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_OD;         // 开漏
            GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10 | GPIO_Pin_11;
            GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
            GPIO_Init(GPIOB, &GPIO_InitStructure);
            GPIO_SetBits(GPIOB, GPIO_Pin_10 | GPIO_Pin_11);          // 释放两根线（总线空闲态=全高）
        }
```

3.3 六个协议函数：把 I2C 时序图”翻译”成代码

起始条件 Start——SCL 高电平期间 SDA 产生下降沿：

```
SDA ──┐       SDA: 1→0（在 SCL=1 时拉低）
      └────
SCL ──────    SCL: 先保证 1，随后拉低 0（占用总线，方便拼接后续时序）
```

```
        void MyI2C_Start(void)
        {
            MyI2C_W_SDA(1);    // 释放 SDA，确保它当前为高（若上一位结束在低，先放上去）

            MyI2C_W_SCL(1);    // 释放 SCL，确保高
            MyI2C_W_SDA(0);    // SCL 高电平期间拉低 SDA → 起始信号
            MyI2C_W_SCL(0);    // 拉低 SCL：①占住总线 ②为发送第一个位做准备（数据在 SCL 低时变化）
        }
```

停止条件 Stop——SCL 高电平期间 SDA 产生上升沿：

```
void MyI2C_Stop(void)
{
    MyI2C_W_SDA(0);   // 先确保 SDA 低
    MyI2C_W_SCL(1);   // 抬 SCL
    MyI2C_W_SDA(1);   // SCL 高时释放 SDA → 停止信号
}
```

发送一字节——高位先行（MSB first），数据在 SCL 低电平期间变化、高电平期间被从机采样：

```
void MyI2C_SendByte(uint8_t Byte)
{
    uint8_t i;
    for (i = 0; i < 8; i ++)
    {
        MyI2C_W_SDA(!!(Byte & (0x80 >> i)));  // 掩码取第 i 位（从最高位起）

        MyI2C_W_SCL(1);                       // 从机在此读走该位
        MyI2C_W_SCL(0);                       // 拉低，允许切换下一位
    }
}
```

`!!(Byte & (0x80 >> i))` 是精髓：`Byte & 0x80` 得到 `0x80` 或 `0x00`（不是 0/1），第一个 `!` 把一切非零变 0、零变 1，第二个 `!` 再翻转回来 → 严格得到 0/1。**发送方永远不读 ACK 之外的 SDA**，全程只管推。

接收一字节——主机交出 SDA 控制权：

```
uint8_t MyI2C_ReceiveByte(void)
{
    uint8_t i, Byte = 0x00;
    MyI2C_W_SDA(1);                    // 释放 SDA！开漏下不释放就读不到从机数据

    for (i = 0; i < 8; i ++)
    {
        MyI2C_W_SCL(1);                          // 时钟高
        if (MyI2C_R_SDA()) {Byte |= (0x80 >> i);} // 采样，按位拼装
        MyI2C_W_SCL(0);
    }
    return Byte;
}
```

应答的两个方向：`SendAck(AckBit)` 主机发应答（0=ACK 继续收，1=NACK 结束收）；`ReceiveAck()` 主机读从机应答（返回 0 表示从机收到了）。两者内部都是”放好 SDA → 打一拍 SCL”。

时序全景（写 MPU6050 一帧）：

```
Start → 1101000+0(0xD0) ACK → 寄存器地址 ACK → 数据 ACK → Stop
```

```
sequenceDiagram
    participant M as MCU(主机)
    participant S as MPU6050(从机)
    M->>S: Start 起始
    M->>S: 0xD0 (地址+写)
    S-->>M: ACK
    M->>S: 寄存器地址(设地址指针)
    S-->>M: ACK
    M->>S: 重复起始 Sr(不释放总线)
    M->>S: 0xD1 (地址+读)
    S-->>M: ACK
    S->>M: 数据字节(前N-1个回ACK)
    M->>S: NACK(最后一个字节)
    M->>S: Stop 终止
```

3.4 MPU6050 寄存器地图

MPU6050_Reg.h 定义了本工程用到的寄存器：

地址	名称	作用	本工程写入值
0x19	SMPLRT_DIV	采样率分频：Rate=内部率/(1+DIV)	0x07 → 1kHz/8=125Hz
0x1A	CONFIG	DLPF 低通滤波配置	0x00 → 滤波最轻
0x1B	GYRO_CONFIG	陀螺量程	0x18 → ±2000°/s（16.4 LSB/°/s）
0x1C	ACCEL_CONFIG	加计量程	0x18 → ±16g（2048 LSB/g）
0x3B~0x40	ACCEL_X/Y/ZOUT_H/L	加速度 3×2 字节	只读
0x41/0x42	TEMP_OUT_H/L	温度	被跳过
0x43~0x48	GYRO_X/Y/ZOUT_H/L	角速度 3×2 字节	只读
0x6B	PWR_MGMT_1	电源管理/时钟源	0x01 → 唤醒+PLL 时钟
0x6C	PWR_MGMT_2	各轴待机	0x00 → 六轴全开
0x75	WHO_AM_I	器件 ID（应读回 0x68）	—

数据寄存器是**连续地址**这一事实，正是”突发读 14 字节”能一次拿全 6 轴的原因。

3.5 MPU6050.c 逐函数精讲

MPU6050.c 的从机地址 `0xD0` = `1101000 << 1 | RW(0)`，即 7 位地址 0x68 + 写标志。

指定地址写一个寄存器：

```
        void MPU6050_WriteReg(uint8_t RegAddress, uint8_t Data)
        {
            MyI2C_Start();
            MyI2C_SendByte(MPU6050_ADDRESS);  MyI2C_ReceiveAck();  // 呼叫从机（写模式）

            MyI2C_SendByte(RegAddress);       MyI2C_ReceiveAck();  // 指定寄存器（地址指
针）
            MyI2C_SendByte(Data);             MyI2C_ReceiveAck();  // 写入数据
            MyI2C_Stop();
        }
```

读寄存器 = 写地址 + 重复起始 + 读：

```
        uint8_t MPU6050_ReadReg(uint8_t RegAddress)
        {
            ...
            MyI2C_Start();                          // ⊦ 重复起始 Sr：不发 Stop 直接再
Start
            MyI2C_SendByte(MPU6050_ADDRESS | 0x01); // 最低位 1 = 读模式
            MyI2C_ReceiveAck();
            Data = MyI2C_ReceiveByte();
            MyI2C_SendAck(1);                       // NACK：只读一个字节，告诉从机到此为止

            MyI2C_Stop();
        }
```

`MPU6050_ADDRESS | 0x01`：0xD0 是”地址+写”，或上 0x01 变 0xD1 即”地址+读”。**为什么用重复起始而不是 Stop+Start?** 读操作要”先写寄存器地址、再读数据”，中间不能让出总线，否则其他主机（这里没有，但协议如此设计）可能插队、从机地址指针被改变。

突发读 14 字节（GetData 的底层）：

```
        void MPU6050_ReadRegs(uint8_t RegAddress, uint8_t *DataArray, uint8_t Count)

        {
            ...同上，重复起始切读模式...
            for (i = 0; i < Count; i ++)
            {
                DataArray[i] = MyI2C_ReceiveByte();
                if (i < Count - 1)  MyI2C_SendAck(0);   // 前 N-1 个字节回 ACK（还要继续）
                else                MyI2C_SendAck(1);   // 最后一个回 NACK（收完了）

            }
            MyI2C_Stop();
        }
```

初始化六连（对应上表）：

```
        void MPU6050_Init(void)
        {
            MyI2C_Init();
            MPU6050_WriteReg(MPU6050_PWR_MGMT_1, 0x01);    // 退出睡眠，时钟选 PLL（比内部 RC 稳）
            MPU6050_WriteReg(MPU6050_PWR_MGMT_2, 0x00);    // 六轴全部不待机
            MPU6050_WriteReg(MPU6050_SMPLRT_DIV, 0x07);    // 采样率 1kHz/(1+7)=125Hz

            MPU6050_WriteReg(MPU6050_CONFIG, 0x00);        // DLPF 配置 0
            MPU6050_WriteReg(MPU6050_GYRO_CONFIG, 0x18);   // 陀螺 ±2000°/s
            MPU6050_WriteReg(MPU6050_ACCEL_CONFIG, 0x18);  // 加计 ±16g
        }
```

量程选择的权衡：量程越大，同样 16 位能分辨的阶梯越粗（灵敏度越低）。±2000°/s 是平衡车常用的”够用不饱和”选择；加计 ±16g 能扛住碰撞冲击不削顶。

GetData 的优化版本：

```
void MPU6050_GetData(int16_t *AccX, ..., int16_t *GyroZ)
{
    uint8_t Data[14];
    MPU6050_ReadRegs(MPU6050_ACCEL_XOUT_H, Data, 14);  // 一次 I2C 读 14 字节

    *AccX = (Data[0] << 8) | Data[1];    // 高字节在前（大端）
    *AccY = (Data[2] << 8) | Data[3];
    *AccZ = (Data[4] << 8) | Data[5];
    //      Data[6]/Data[7] 是温度，跳过
    *GyroX = (Data[8] << 8) | Data[9];
    *GyroY = (Data[10] << 8) | Data[11];
    *GyroZ = (Data[12] << 8) | Data[13];
}
```

对比注释掉的旧版（6 次单字节读，每次完整 Start/地址/地址指针）：新版**一次事务拿全**，I2C 开销降为 1/6。中断里每 10ms 调用一次，这笔账不小。`int16_t` 接住 `(H<<8)|L` 自动获得补码语义（负角速度直接是负数）。

3.6 互补滤波系数 0.01 的推导

融合公式按周期展开：

```
Angle[k] = 0.01·AngleAcc[k] + 0.99·(Angle[k-1] + ω·T)
```

把它看成两个滤波器：- 对陀螺通路：`y[k] = 0.99·(y[k-1] + x[k]·T)`——一阶高通（直流漂移每周期衰减 1%）；- 对加计通路：`y[k] = 0.01·x[k] + 0.99·y[k-1]`——一阶低通，截止频率 $f_c \approx \frac{0.01}{2\pi \cdot 0.01s} \approx 0.16\,\text{Hz}$，时间常数 τ ≈ 0.99s。

物理图像：**1 秒以内信陀螺**（车身晃动、控制动作都发生在这个尺度），**1 秒以上信加计**（把陀螺的漂移慢慢拉回真值）。α 取太大会跟着加计抖，取太小漂移收敛慢——0.01 是平衡车的经典取值。

本章思考：1. 为什么读一个寄存器要用重复起始，而不能 Stop 后重新 Start 写地址再读？2. 若陀螺量程改成 ±500°/s，`GY/32768*2000` 这行要怎么改？零偏 16 还成立吗？3. 突发读为什么”前 N−1 个 ACK、最后一个 NACK”？如果全部 NACK 会发生什么？

第 4 章 执行机构：PWM / Motor / Encoder

4.1 PWM.c：20kHz 调速的诞生过程

PWM.c 用 TIM2 的 CH1/CH2 输出两路 PWM（PA0/PA1）。

频率计算逐步演示：

```
计数时钟 = 72MHz
PSC = 36-1      → 计数频率 = 72MHz / 36 = 2MHz（每 0.5μs 计一个数）
ARR = 100-1     → 从 0 数到 99 共 100 步，溢出即一个 PWM 周期
PWM 频率  = 2MHz / 100 = 20kHz ✓（人耳听不见，电机不啸叫）
PWM 分辨率 = 100 级（对应代码里的 PWM 值域 0~99）
```

为什么是 20kHz? 太低（<16kHz）电机会发出可闻啸叫；太高开关损耗大、MOS 驱动发热。20kHz 是直流电机调速的黄金频率。

```
        GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;    // 复用推挽：引脚控制权交给片上外设 TIM2
```

PWM1 模式（`TIM_OCMode_PWM1`）：`CNT < CCR` 时输出有效电平（高），`CNT ≥ CCR` 输出无效。所以 **CCR 越大占空比越大**——`PWM_SetCompare1(x)` 的 x 就是占空比 = x%。库函数 `TIM_SetCompare1` 仅一行：写 CCR1 寄存器，随时可改、下个周期生效。

`TIM_OCStructInit(&...)` 先填默认值再改三项，避免结构体残留栈垃圾（标准库的惯用法）。

4.2 Motor.c：有符号速度 → H 桥方向 + 占空比

直流电机换向靠 **H 桥**（板上驱动芯片如 TB6612/DRV8833）：IN1=1、IN2=0 正转；IN1=0、IN2=1 反转；IN1=IN2=0 滑行。PWMA/PWMB 引脚上的 PWM 决定速度。

```
        void Motor_Init(void)
        {
            RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
            GPIO_InitTypeDef GPIO_InitStructure;
            GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;   // 方向脚用普通推挽输出

            GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12 | GPIO_Pin_13 | GPIO_Pin_14 |
GPIO_Pin_15;
            ...
            PWM_Init();     // PWM 通道随后就绪
        }
```

```
        void Motor_SetPWM(uint8_t n, int8_t PWM)   // 注意入参是 int8_t：-128~127

        {
            if (n == 1)
            {
                PWM = -PWM;                          // ⊢ 镜像补偿
                if (PWM >= 0)     // 定义"正转"方向
                {
                    GPIO_SetBits(GPIOB, GPIO_Pin_12);      // IN1=1
                    GPIO_ResetBits(GPIOB, GPIO_Pin_13);    // IN2=0
                    PWM_SetCompare1(PWM);
                }
                else
                {
                    GPIO_ResetBits(GPIOB, GPIO_Pin_12);    // IN1=0
                    GPIO_SetBits(GPIOB, GPIO_Pin_13);      // IN2=1
                    PWM_SetCompare1(-PWM);                 // 取负得绝对值再给 CCR


                }
            }
            else if (n == 2) { /* 同构，Pin14/15 + CCR2 */ }
        }
```

两个要点：

1. `PWM = -PWM` **镜像补偿**：左右电机在车上对脸安装，同样的”电正转”一个让车向前一个让车向后。这里对两个电机入参都取负，把”轮子物理正转”统一成”车向前”。装到哪侧、线怎么接，都折算进这一行——上层永远只说”向前 50”。

2. **int8_t 的隐藏风险：** `Motor_SetPWM` 入参是 `int8_t`，而中断里传的是 `int16_t LeftPWM`——隐式截断！好在限幅 ±100 在传参前已完成，100 < 127 安全。**若哪天改了限幅上限超过 127，这里会悄悄溢出翻转**，属于需要警惕的接口约束。

4.3 Encoder.c：正交编码与四倍频

原理：A/B 两相 90° 相位差

带磁环编码器的电机输出 A、B 两路脉冲。正转时 A 超前 B 90°，反转时 B 超前 A 90°——**相位关系编码了方向，边沿数量编码了距离**。

```
flowchart LR
    subgraph FWD ["正转：A 超前 B 90°"]
        direction LR
        A1["A↑ +1"] --> A2["B↑ +1"] --> A3["A↓ +1"] --> A4["B↓ +1"]
    end
    subgraph REV ["反转：B 超前 A 90°"]
        direction LR
        B1["B↑ −1"] --> B2["A↑ −1"] --> B3["B↓ −1"] --> B4["A↓ −1"]
    end
    FWD -- "每个边沿都计数 → 4倍分辨率" --> CNT["计数器 CNT 增/减"]
    REV --> CNT
```

“TI12 四倍频”（`TIM_EncoderMode_TI12`）：计数器在 **A 的上升沿、A 的下降沿、B 的上升沿、B 的下降沿**都计数一次。一圈的计数从”每相脉冲数”变为 4 倍，分辨率提高 4 倍；计数增减方向由硬件根据 A/B 相位自动判断——正转递增、反转递减，**软件完全不参与辨向**。

初始化逐行

```
        void Encoder_Init(void)
        {
            /*TIM3 → 左轮，PA6/PA7*/
            RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
            RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

            GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;    // 上拉输入：编码器集电极开路输出需要
            GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;
            ...

            TIM_TimeBaseInitStructure.TIM_Period = 65536 - 1;     // ARR=65535：16位计数器用满
            TIM_TimeBaseInitStructure.TIM_Prescaler = 1 - 1;      // PSC=0：每个边沿计一次，不分频
            ...
            TIM_ICInitStructure.TIM_ICFilter = 0xF;    // 输入滤波最大档！
            TIM_ICInit(TIM3, &TIM_ICInitStructure);    // CH1、CH2 各配一次

            TIM_EncoderInterfaceConfig(TIM3, TIM_EncoderMode_TI12,
                                       TIM_ICPolarity_Rising,
TIM_ICPolarity_Rising); // 编码器模式
            TIM_Cmd(TIM3, ENABLE);
            /*TIM4 → 右轮，PB6/PB7，配置完全同构*/
        }
```

细节深挖：

- **ARR=65535 的妙处**：计数器 16 位回绕。倒转 1 格：0 → 65535。用 `int16_t` 接住 65535，补码正好是 **−1**——硬件回绕与 C 有符号溢出完美互译，加减小增量时天然正确，无需软件处理跨零。
- **`TIM_ICFilter = 0xF`**：输入捕获滤波器最强档。数字滤波器按 f_DTS 时钟连续采样 N 次（0xF 对应 N=8）全一致才认定有效边沿，过滤电机碳火花/线缆干扰带来的毛刺。毛刺若被误计，速度环反馈就是假的——**编码器滤波宁强勿弱**。
- 编码器模式下 CNT 的方向、时钟全部由硬件接管，`TIM_CounterMode_Up` 等时基参数实际不生效，但库要求填完整结构体。

读取：读后清零

```
        int16_t Encoder_Get(uint8_t n)
        {
            int16_t Temp;
            if (n == 1) { Temp = TIM_GetCounter(TIM3); TIM_SetCounter(TIM3, 0); retur
n Temp; }
            else if (n == 2) { Temp = TIM_GetCounter(TIM4); TIM_SetCounter(TIM4, 0);
return Temp; }
            return 0;
        }
```

读走计数 → 立即清零 → 返回值就是**自上次读取以来的增量**。配合 50ms 的固定读取周期，增量/0.05 = 速度。为什么用增量而不是累计值？累计值会无限增大（64 位才够），且”清零取增量”天然免疫上电时轮子的历史位置。

第 5 章 系统节拍：Timer / Delay / 中断系统

5.1 Timer.c：1kHz 心跳

Timer.c 配置高级定时器 TIM1：

```
PSC = 72-1   → 计数时钟 = 72MHz/72 = 1MHz（1μs 一格）
ARR = 1000-1 → 数 1000 格溢出一次 → 1ms，即 1kHz 更新中断
```

```
        TIM_ClearFlag(TIM1, TIM_FLAG_Update);   // ⊢ 关键一行
        TIM_ITConfig(TIM1, TIM_IT_Update, ENABLE);
```

为什么先清标志再开中断？ `TIM_TimeBaseInit` 写 PSC/ARR 时会产生一次”更新事件”，把 UIF 标志置上了。若不清除，`TIM_ITConfig` 一使能，NVIC 立刻响应这个”陈旧中断”——初始化代码还没返回就跳进了中断服务。标准流程：**清标志 → 开中断 → 开定时器**。

NVIC 优先级配置：

```
        NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);  // 2 位抢占 + 2 位响应
        NVIC_InitStructure.NVIC_IRQChannel = TIM1_UP_IRQn;
        NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 2;   // 抢占 2
        NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;          // 响应 1
```

全工程优先级格局（数字越小优先级越高）：

中断	抢占	响应	说明
USART1	1	1	调试串口接收
USART2	1	1	蓝牙接收
TIM1_UP	**2**	1	1kHz 控制节拍

TIM1 的抢占数字最大 = 抢占优先级最低，意味着串口中断可以打断 10ms 姿态计算。这是合理的：串口 ISR 只收几个字节（几 μs），打断 1ms 预算无伤；反过来若 TIM1 抢占级最高，控制中断会延迟串口接收，9600 波特率下 1ms 只有约 1 个字节的余量，容易丢字节。**短中断让长中断**是优先级设计的基本功。

> 函数名 `TIM1_UP_IRQHandler` 必须与启动文件 `startup_stm32f10x_md.s` 向量表中的弱定义名字逐字一致，否则中断发生时跳到默认死循环 `B .`，现象是”程序像卡死”。

5.2 Delay.c：不占用定时器外设的 SysTick 延时

```
        void Delay_us(uint32_t xus)
        {
            SysTick->LOAD = 72 * xus;              // 72MHz 下 1μs = 72 个时钟
            SysTick->VAL = 0x00;                   // 写 VAL 任意值都会清计数器
            SysTick->CTRL = 0x00000005;            // bit0 ENABLE=1 启动 | bit2
CLKSOURCE=1 用 HCLK
            while(!(SysTick->CTRL & 0x00010000));  // 轮询 bit16 COUNTFLAG（读到 1 自动
清零）
            SysTick->CTRL = 0x00000004;            // ENABLE=0 关闭，省电
        }
```

SysTick 是 Cortex-M 内核自带的 24 位递减计数器，**不属于 STM32 外设，不占用 TIM1~TIM4**。要点：

- `0x00010000` 是第 16 位 COUNTFLAG：计到 0 硬件置 1，读取后自动清零——一行轮询搞定；
- 最大延时 = $2^{24}/72 \approx$ **233015μs**（LOAD 是 24 位寄存器），所以头文件注明范围；
- `Delay_ms` 用循环叠 1000 次 `Delay_us`，避开 24 位上限，也避免大数乘法误差。

本工程把 Delay 用于 OLED 上电稳定延时与 I2C 备用时序，**不用于主流程**（实时系统忌讳忙等）。

本章思考： 1. 把 TIM1 优先级改成抢占 0（最高），可能出现什么新问题？哪个场景下反而应该这么做？ 2. `Delay_us(234000)` 会发生什么？为什么？

第 6 章 人机交互：Key / LED / OLED

6.1 Key.c：中断驱动的状态机消抖

机械按键按下/释放瞬间触点弹跳，电平在几 ms 内抖动多次，GPIO 直接读会把一次按下读成多次。本工程用**定时采样 + 状态机**，彻底不依赖阻塞延时。

```
        uint8_t Key_GetState(void)      // 纯读硬件：当前哪个键被按着（低电平=按下，上拉输入）

        {
            if (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_1) == 0) return 1;
            if (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_0) == 0) return 2;
            if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_5) == 0) return 3;
            if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_4) == 0) return 4;
            return 0;
        }
```

```c
void Key_Tick(void)            // 被 1kHz 中断每 ms 调一次
{
    static uint8_t Count;
    static uint8_t CurrState, PrevState;

    Count ++;
    if (Count >= 20)           // 每 20ms 采样一次 → 消抖
    {
        Count = 0;
        PrevState = CurrState;
        CurrState = Key_GetState();
        if (CurrState == 0 && PrevState != 0)   // 上次有键、这次无键 = 释放沿

        {
            Key_Num = PrevState;              // 记下"刚释放的是几号键"
        }
    }
}

uint8_t Key_GetNum(void)       // 主循环取键值，读后清零
{
    uint8_t Temp;
    if (Key_Num) { Temp = Key_Num; Key_Num = 0; return Temp; }
    return 0;
}
```

三层设计各有深意：

1. **20ms 采样间隔即消抖**：抖动只发生在按下/释放后的 5~10ms 内，每 20ms 看一眼，抖动期正好被跳过。这叫”定时采样消抖”，与”延时后重读确认”效果相同但零阻塞。
2. **释放沿触发**（`CurrState==0 && PrevState!=0`）：按住不放只触发**一次**。如果用按下沿，长按 1 秒会被 50 次 20ms 采样误读为……不会（状态机只记边沿），但释放沿的真正好处是**手指离开的瞬间才生效**，给人”确认感”，且避免按下瞬间手抖的误触发。
3. **中断里喂 Tick、主循环取结果**：把”看门”的活放进 1kHz 中断（时基绝对准确，不受主循环 OLED 刷新十几 ms 的阻塞影响），取键值用”读后清零”队列语义。`Key_GetNum` 在主循环调用，与 `Key_Num`（中断写）是单字节原子访问，无需关中断保护。

6.2 LED.c：三行代码 + 灌电流常识

PC13 推挽输出，板载 LED 接 VCC→电阻→LED→PC13，所以**低电平点亮**（灌电流，STM32 GPIO 灌电流能力 20mA 强于拉电流）：

```
void LED_ON(void)  { GPIO_ResetBits(GPIOC, GPIO_Pin_13); }   // 拉低=亮
void LED_OFF(void) { GPIO_SetBits(GPIOC, GPIO_Pin_13); }     // 拉高=灭
```

`LED_Turn()` 用 `GPIO_ReadOutputDataBit` 读**输出寄存器**（不是输入），实现电平翻转。

6.3 OLED.c：一款完整显示驱动的解剖

OLED.c（约 1480 行）驱动 SSD1306 控制器的 128×64 点阵屏，架构是经典的”**显存 + 绘图 API + 批量刷屏**”三层：

```
应用层   OLED_Printf / ShowNum / DrawLine ...
              │  只读写显存（内存操作，μs 级）
显存层   OLED_DisplayBuf[8][128]  ← 128×64 屏的位图镜像，1KB
              │  OLED_Update() 时才批量发硬件
传输层   软件 I2C（PB8=SCL，PB9=SDA，从机地址 0x78）
```

```
flowchart TD
    A["应用层<br>OLED_Printf / ShowNum / DrawLine …"] -- "只读写显存(μs级)" -->
B["显存层<br>OLED_DisplayBuf[8][128] = 1KB"]
    B -- "OLED_Update() 批量刷屏" --> C["传输层<br>软件I2C PB8/PB9 · 地址0x78"]
    C --> D["SSD1306 控制器<br>128×64 点阵屏"]
```

6.3.1 显存的”页”结构（理解一切显示函数的钥匙）

SSD1306 按”页”组织显存：**1 页 = 8 行像素 = 1 字节/列**。64 行 = 8 页。每个字节 bit0 在上、bit7 在下：

```
       列0      列1      ...  列127
页0   [B0B1..B7][...]          ← 字节高位在下！
页1   [...]
...
页7   [...]
```

任意像素 (X, Y) 的操作都是两步定位：

```
        OLED_DisplayBuf[Y / 8][X] |= 0x01 << (Y % 8);    // 画点（DrawPoint 核心）

        OLED_DisplayBuf[Y / 8][X] &= ~(0x01 << (Y % 8)); // 擦点（ClearArea 核心）
```

6.3.2 通信层：控制字节是 SSD1306 的”暗号”

```
        void OLED_WriteCommand(uint8_t Command)
        {
            OLED_I2C_Start();
            OLED_I2C_SendByte(0x78);      // 从机地址（SA0=0 时的 8 位写地址）
            OLED_I2C_SendByte(0x00);      // 控制字节 0x00：后面是命令
            OLED_I2C_SendByte(Command);
            OLED_I2C_Stop();
        }
        void OLED_WriteData(uint8_t *Data, uint8_t Count)
        {
            ...
            OLED_I2C_SendByte(0x40);      // 控制字节 0x40：后面全是显存数据，可连续发 N 字
节
            for (i = 0; i < Count; i++) OLED_I2C_SendByte(Data[i]);
            ...
        }
```

同一根 I2C 上”命令”与”数据”靠第一字节的控制位区分（Co/D/C 位）。数据可以一口气连发——这正是 `OLED_Update` 效率的来源。

6.3.3 初始化：16 条命令点亮一块屏

```
        void OLED_Init(void)
        {
            OLED_GPIO_Init();                 // 开漏配置 + 1000×1000 空循环等电源稳

            OLED_WriteCommand(0xAE);          // 先关显示（防配置过程花屏）
            OLED_WriteCommand(0xD5); OLED_WriteCommand(0x80);  // 显示时钟分频
            OLED_WriteCommand(0xA8); OLED_WriteCommand(0x3F);  // 复用率 63 → 64 行

            OLED_WriteCommand(0xD3); OLED_WriteCommand(0x00);  // 显示偏移 0
            OLED_WriteCommand(0x40);          // 显示起始行 0
            OLED_WriteCommand(0xA1);          // 段重映射：列 127→SEG0（正常视角左右正确）

            OLED_WriteCommand(0xC8);          // COM 扫描方向：从下到上（上下正确）

            OLED_WriteCommand(0xDA); OLED_WriteCommand(0x12);  // COM 引脚配置（Alt 模
式）
            OLED_WriteCommand(0x81); OLED_WriteCommand(0xCF);  // 对比度 0xCF
            OLED_WriteCommand(0xD9); OLED_WriteCommand(0xF1);  // 预充电周期
            OLED_WriteCommand(0xDB); OLED_WriteCommand(0x30);  // VCOMH 电平
            OLED_WriteCommand(0xA4);          // 显示跟随显存（非全亮）
            OLED_WriteCommand(0xA6);          // 正常显示（非反色）
            OLED_WriteCommand(0x8D); OLED_WriteCommand(0x14);  // ⊦ 开内部电荷泵！

            OLED_WriteCommand(0xAF);          // 开显示
            OLED_Clear(); OLED_Update();      // 清一次屏，防上电随机数据花屏
        }
```

0x8D/0x14 充电泵最关键：4 针 OLED 模块没有外部高压源，SSD1306 靠内部电荷泵把 3.3V 升到驱动 OLED 的 ~12V，不开启则屏完全不亮——这是”OLED 不显示”问题的头号原因。0xA1/0xC8 若配错，图像上下/左右镜像。

光标设置用三条命令拼一个 16 位地址：

```
        void OLED_SetCursor(uint8_t Page, uint8_t X)
        {
            OLED_WriteCommand(0xB0 | Page);                    // 页地址
            OLED_WriteCommand(0x10 | ((X & 0xF0) >> 4));       // 列地址高 4 位
            OLED_WriteCommand(0x00 | (X & 0x0F));              // 列地址低 4 位
        }
```

注释里保留的 `X += 2`：1.3 寸屏用 SH1106 控制器、显存 132 列且屏幕错位 2 列，驱动 1.3 寸屏必须解除注释——同一份代码兼容两种屏的开关。

6.3.4 刷屏与文本渲染

```
        void OLED_Update(void)      // 全屏刷新：8 页 × 每页连发 128 字节
        {
            uint8_t j;
            for (j = 0; j < 8; j ++)
            {
                OLED_SetCursor(j, 0);
                OLED_WriteData(OLED_DisplayBuf[j], 128);   // 连续数据模式一次发一页

            }
        }
```

字符显示的本质是**查字模库 + 搬运到位图**：

```
        void OLED_ShowChar(int16_t X, int16_t Y, char Char, uint8_t FontSize)
        {
            if (FontSize == OLED_8X16)
                OLED_ShowImage(X, Y, 8, 16, OLED_F8x16[Char - '
']);   // ASCII 减空格 = 库下标
            else if (FontSize == OLED_6X8)
                OLED_ShowImage(X, Y, 6, 8, OLED_F6x8[Char - ' ']);
        }
```

`Char - ' '`：字模库从空格（ASCII 32）开始连续存到 `~`（126），任意字符下标 = ASCII − 32。

OLED_ShowString 的多字节字符解析（支持中文）值得一提：UTF-8 模式下按首字节高位模式判字长——

```
        if      ((String[i] & 0x80) == 0x00) CharLength = 1;   // 0xxxxxxx   ASCII

        else if ((String[i] & 0xE0) == 0xC0) CharLength = 2;   // 110xxxxx   2 字节

        else if ((String[i] & 0xF0) == 0xE0) CharLength = 3;   // 1110xxxx   3 字节（中
文在此）
        else if ((String[i] & 0xF8) == 0xF0) CharLength = 4;   // 11110xxx   4 字节
```

提取出完整字符后到 `OLED_CF16x16` 自定义中文字库线性查找（`strcmp` 匹配 Index 字符串），找到则 16×16 显示，找不到显示”方框问号”。**源码编码必须与 OLED.h 中定义的 `OLED_CHARSET_UTF8` / `OLED_CHARSET_GB2312` 一致**，否则中文全是乱码——Keil 默认 GB2312，VSCode 默认 UTF-8。

数字显示用”幂取位法”（与 Serial_SendNumber 同款）：

```
        OLED_ShowChar(X + i * FontSize, Y, Number / OLED_Pow(10, Length - i - 1) %
10 + '0', FontSize);
        //                  除以 10^(第i位的权重) %10 = 提取第 i 位数字, +'0' 转成字符
```

`OLED_ShowFloatNum` 多一步**四舍五入进位**：`FraNum = round(小数×10^FraLength)`，若舍入后 `FraNum/PowNum` 进位（0.999→1.00），要把 1 加回整数部分——细节决定显示是否”跳变”。

`OLED_Printf` 用 `va_list/vsprintf` 在 256 字节栈缓冲里完成格式化再交给 ShowString——注意**格式串超长会栈溢出**，%05.2f 这类要节制。

6.3.5 图像搬运：OLED_ShowImage 的跨页位移

字模数据按”从第 0 页开始”的布局存储，但显示位置 Y 不一定是 8 的倍数——需要把一个字节**拆到两页**：

```
        OLED_DisplayBuf[Page + j][X + i]     |= Image[j * Width + i] << (Shift);
// 高位留在本页
        OLED_DisplayBuf[Page + j + 1][X + i] |= Image[j * Width + i] >> (8 - Shift);
// 低位溢到下页
```

`Shift = Y % 8`。这就是所有”非 8 对齐坐标”都能显示的原因。函数开头 `OLED_ClearArea` 先擦除目标区域，保证透明背景（不残留旧像素）。

6.3.6 绘图算法：Bresenham 一家子

- **DrawLine**：Bresenham 直线算法——全程整数加减，用决策变量 d 判定”下一个点走 E 还是 NE”，避免浮点和乘除。实现先把任意象限的线段**归一化**到”第一象限、斜率 0~1”：x 反向→交换两端点（yflag 不需要）；y 反向→坐标取负+`yflag`；斜率>1→xy 互换 +`xyflag`；画点时按标志位把坐标**变换回去**。这套”归一化→算法→反变换”的套路值得背下来。
- **DrawCircle**：Bresenham 画圆 + **八分对称性**——算出 1/8 圆弧的点 (x,y)，其余 7 个对称点 `(±x,±y)/(±y,±x)` 免费获得，效率 ×8。决策变量 `d = 1-R` 起步，`d<0` 加 `2x+1`（走东点），否则加 `2(x−y)+1`（走东南点）。

- **DrawEllipse**：椭圆曲率不均，分”靠长短轴端点的两段”用两个决策变量 d1/d2 各自递推。
- **DrawTriangle（填充）**：包围盒内逐点调用 `OLED_pnpoly`（W. Randolph Franklin 射线法）判点在多边形内——**射线法**：从测试点向右发射线，穿越多边形边奇数次即在内部。代码里 `c = !c` 就是”穿越计数取反”。
- **DrawArc**：复用画圆的八分点生成，每个候选点用 `OLED_IsInAngle`（atan2 求点角度 + 环形区间判断）过滤。

6.3.7 OLED_Data.c：字模库

存三个数组：`OLED_F8x16[95][16]`、`OLED_F6x8[95][6]`（ASCII 95 个可见字符）、`OLED_CF16x16[]`（中文自定义库，每项 = 字符串 Index + 32 字节点阵）。字模由取模软件生成，**纵向取模、字节倒序**（与 6.3.1 的页结构对应）。想加新汉字：取模软件生成 16×16 数据，按格式追加一项即可。

本章思考： 1. 为什么 OLED_ShowString 显示 GB2312 中文只需判断最高位，UTF-8 却要判三位？ 2. OLED_Update 全屏要 ~10ms，若把它也放进 1kHz 中断里调用会发生什么？

第 7 章 串行通信：Serial / BlueSerial

工程里有两路几乎同构的串口驱动，职责不同：

	Serial（USART1, PA9/PA10）	BlueSerial（USART2, PA2/PA3）
用途	调试打印（printf 重定向）	蓝牙模块收发调参/遥控指令
接收方式	单字节 + RxFlag	**状态机整包提取** `[...]`
关键全局量	Serial_RxData / Serial_RxFlag	BlueSerial_RxPacket[100] / BlueSerial_RxFlag

7.1 初始化配置逐项解读

```
        USART_InitStructure.USART_BaudRate = 9600;                                  // 波特
率
        USART_InitStructure.USART_HardwareFlowControl =
USART_HardwareFlowControl_None;  // 无流控
        USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;         // 收发
都开
        USART_InitStructure.USART_Parity = USART_Parity_No;                     // 无校
验
        USART_InitStructure.USART_StopBits = USART_StopBits_1;                  // 1 位
停止位
        USART_InitStructure.USART_WordLength = USART_WordLength_8b;             // 8 位
数据
```

即标准 **9600-8-N-1**，必须与对端（USB 转串口工具 / 蓝牙模块默认配置）完全一致。引脚方向是 USART 的固定套路：**TX 配复用推挽（AF_PP）**——数据由外设推出去；**RX 配上拉输入（IPU）**——空闲电平为高，符合 UART 空闲态。

中断配置与 Timer 同款（NVIC 组 2，USART1/USART2 都在抢占 1，可打断 TIM1 控制中断，理由见 5.1）。

7.2 发送：从 SendByte 到 printf

```
        void Serial_SendByte(uint8_t Byte)
        {
            USART_SendData(USART1, Byte);      // 写 TDR，硬件自动移位发出
            while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);   // 等
TXE=1
        }
```

TXE 语义：发送数据寄存器空（数据已搬入移位寄存器）。写 DR 后等 TXE，保证下一个字节到来时上一个已让位——这是”发送后等待”的写法；“发送前等待”（先等 TXE 再写）亦可。注意这里没有等 TC（发送完成），函数返回时最后一位可能还在移位中，连续 SendByte 无影响，但**发完立刻进低功耗/复位会截断数据**。

printf 重定向只需一个函数（Keil MicroLIB）：

```c
int fputc(int ch, FILE *f)
{
    Serial_SendByte(ch);
    return ch;
}
```

此后 `printf("Angle=%f\r\n", Angle);` 直接走串口。自己实现的 `Serial_Printf` 则用 `va_list + vsprintf` 先格式化到 100 字节缓冲再整串发出（不依赖重定向，逻辑更可控）。

`Serial_SendNumber` 用幂取位法把 uint32 逐位转 ASCII（`%10 + '0'`），同 OLED_ShowNum 思路，不再展开。

7.3 BlueSerial 的包接收状态机（重点）

串口是**字节流**，没有”消息边界”。App 发的指令 `[slider,AngleKp,5.5]` 到达 MCU 时只是一个字节一个字节的中断。如何拼回”一包”？本工程用两状态状态机，以 `[` 开包、`]` 闭包：

```
        void USART2_IRQHandler(void)
        {
            static uint8_t RxState = 0;      // 0=等包头  1=收包体
            static uint8_t pRxPacket = 0;    // 包内写指针
            if (USART_GetITStatus(USART2, USART_IT_RXNE) == SET)
            {
                uint8_t RxData = USART_ReceiveData(USART2);   // 读 DR 同时清 RXNE 标
志

                if (RxState == 0)                     // 状态0：等包头
                {
                    if (RxData == '[' && BlueSerial_RxFlag == 0)   // ⊦ Flag=1 期间拒绝
新包
                    {
                        RxState = 1;
                        pRxPacket = 0;                // 复位写指针
                    }
                }
                else if (RxState == 1)                // 状态1：收包体
                {
                    if (RxData == ']')                // 闭包
                    {
                        RxState = 0;
                        BlueSerial_RxPacket[pRxPacket] = '\0';   // 补字符串结束符

                        BlueSerial_RxFlag = 1;                   // 通知主循环
                    }
                    else
                    {
                        BlueSerial_RxPacket[pRxPacket] = RxData; // 逐字节入缓冲

                        pRxPacket ++;
                    }
                }
                USART_ClearITPendingBit(USART2, USART_IT_RXNE);
            }
        }
```

逐个设计点：

1. `BlueSerial_RxFlag == 0` **才允许开新包**：主循环还在解析上一包时，新指令整包丢弃。牺牲实时性换**数据一致性**——比”解析到一半缓冲被覆盖”安全得多。
2. **静默丢弃包外字符**：状态 0 下非 `[` 的字节（干扰、残包尾巴）直接忽略，天然抗噪。

3. **隐患：无长度保护**：若发来 `[` 后没有 `]` 且长度 >100，`pRxPacket` 越界写坏相邻内存。可信环境下可接受，工程化应加 `if (pRxPacket >= 99) {RxState=0;}` 防御。改进时也可以加**包间超时**：收到 `[` 后 100ms 无 `]` 则复位状态机，防止半包卡死接收。

7.4 主循环侧的协议落地

第 1 章已详解 `strtok` 三段式解析，这里补一张协议总表：

包格式	字段	作用
`[slider,Name,Value]`	Name ∈ {AngleKp, AngleKi, AngleKd, SpeedKp…, TurnKd, Offset}	atof 写入对应 PID 参数 / OutOffset
`[joystick,LH,LV,RH,RV]`	四个 int8 摇杆值	SpeedPID.Target = LV/25；TurnPID.Target = RH/25
`[key,Name,Action]`	预留	函数体为空，待扩展

`BlueSerial_Printf("[plot,%f,%f,%f]", ...)` 被注释的那行：原本把三个积分器值发回上位机画曲线——**PID 调参的”示波器”**。建议调参阶段打开它，配合 VOFA+ 等上位机看 ErrorInt 的收敛情况。

第 8 章 工程组织与其它文件

8.1 stm32f10x_it.c：空的中断模板

此文件只包含系统异常（NMI/HardFault 等）的空模板。**本工程的中断服务函数写在了各自相关文件里**（TIM1_UP_IRQHandler 在 main.c、USART1_IRQHandler 在 Serial.c、USART2_IRQHandler 在 BlueSerial.c）——C 语言的链接器只关心全局符号名，不关心函数写在哪个 .c 文件，只要与启动文件向量表里的弱符号名一致即可覆盖。这种”ISR 跟着模块走”的组织方式让每个驱动自包含。

8.2 头文件的组织规则

每个驱动头文件的标准三件套：

```
#ifndef __PID_H      // ① include guard：防重复包含（编译期只展开一次）
#define __PID_H

typedef struct {...} PID_t;   // ② 对外暴露的类型
void PID_Init(PID_t *p);      // ③ 对外暴露的函数原型
void PID_Update(PID_t *p);

#endif
```

- **.h 只放声明，.c 放实现**；`static` 函数/变量（如 OLED 工具函数、Key_Tick 里的 static 局部量）只在 .c 内可见——这就是 C 的”私有”。
- 各 .h 之间用 `#include` 显式声明依赖（如 OLED.h include OLED_Data.h），不依赖”恰好被别的头带进来”。

8.3 stm32f10x_conf.h：标准库的总闸

```
//#include "stm32f10x_adc.h"     // 用不到的外设库可以注释掉，减编译体积
#include "stm32f10x_gpio.h"
#include "stm32f10x_tim.h"
#include "stm32f10x_usart.h"
#include "misc.h"                // NVIC
...
```

同时控制 `assert_param` 宏：未定义 `USE_FULL_ASSERT` 时断言为空——发布版零开销。

第 9 章 调参方法与常见问题 FAQ

9.1 调参顺序（血泪总结）

第 0 步：静态验证（车架空，RunFlag=0） - 屏上 GY 读数：静止时应约 ±5 以内，否则改 `GY -= 16` 零偏； - 手倾车身：Angle 应平滑跟随、方向正确（前倾为正还是负要与代码约定一致）； - 手转轮子：AveSpeed/DifSpeed 变化正确，同转应只动 AveSpeed。

第 1 步：只调直立环（Ki=Ki=0.1 先置 0，速度环/转向环不启用） 1. 增大 Kp 到车微微低频晃动 → 回退到 60%； 2. 增大 Kd 阻尼高频抖动，到车”稳如被手扶”； 3. 最后加小 Ki 消除缓慢漂移（Ki 过大又会低频摇摆）。

第 2 步：合上速度环 - Kp 从小加起：响应是”推车它顶回来”；过大表现为前后大幅慢摆； - Ki 消除缓慢匀速漂移（受地面坡度/死区影响）。

第 3 步：转向环：Kp 决定转向跟随快慢，Ki 决定能否走直线。

每一步都用蓝牙 slider 在线改参数 + OLED 三列仪表观察，这就是这套程序调试架构的价值。

9.2 症状速查表

症状	可能原因	处置
一启动就朝一个方向冲刺到倒	电机/传感器/编码器任一符号反了	核对 `AvePWM=-Out`、`PWM=-PWM`、GY 方向
静止时 Angle 缓慢单向漂	陀螺零偏变了	重标 `GY-=16`（或上电自动校准）
静止时 Angle 有固定小偏差	机械安装不水平	调 `AngleAcc += 0.5` 机械零点
高频嘶嘶抖	直立环 Kp 过大 / Kd 不足	降 Kp 或升 Kd
低频大幅摇摆	速度环 Kp 过大或直立环 Ki 过大	减小相应参数
起步猛窜一下	没清积分器 / Target 残留	确认启动前 `PID_Init` 三个环
小信号不动、加一点又太猛	电机死区	调 `OutOffset`（先小后大）
OLED 不亮/花屏	充电泵未开 / I2C 上拉缺失 / 供电不足	核对 0x8D 0x14、4.7k 上拉、独立 3.3V
蓝牙收不到	波特率不符 / TX-RX 未交叉 / 括号包格式错	9600、TX↔RX 对调、`[...]` 格式
遥控半程就反向	App 发 0~255 而 int8_t 解析	上位机改发 -128~127
中断偶发超时	OLED/调试打印混入中断	`TimerErrorFlag` 定位，重活移回主循环

9.3 实时性账本（中断预算 1ms）

操作	耗时量级
MPU6050 突发读 14 字节（软 I2C）	~200-400μs（大头）
PID×3 + 角度解算（含 atan2）	~几十 μs
Encoder_Get ×2	<2μs
全部 1ms 分支（Key_Tick 等）	<5μs

10ms 分支合计约 0.3~0.5ms，预算内但**不宽裕**——若把 OLED_Update（~10ms）放进去必然超时，这就是它待在主循环的原因。想缩短：给 MPU6050 换硬件 I2C 或提高软 I2C 速率。

第 10 章 思考题与进阶方向

10.1 各章思考题汇总

1. （第1章）`Timer_Init` 提前会怎样？——中断读未初始化外设，I2C 卡死/垃圾数据。
2. （第1章）机械零点 0.5 如何重测？——静止放平，读 Angle 显示值，把它的相反数加进 `AngleAcc +=`。
3. （第2章）为什么直立环必须 D 项而速度环可以不要？——内环对象快且欠阻尼需微分镇定；外环 50ms+机械惯性本身即滤波。
4. （第3章）重复起始的必要性？——读写切换期间必须保持总线控制权与寄存器指针。
5. （第4章）编码器 ARR 为什么是 65535 而不是更小？——用满 16 位 + int16 补码天然处理回绕符号。
6. （第5章）串口抢占级为何高于 TIM1？——串口 ISR 极短、字节到达不可延迟；控制环可容忍 μs 级抖动。
7. （第6章）释放沿 vs 按下沿触发的体验差异？——释放沿有”确认感”，长按不重复。
8. （第7章）状态机丢包保护的代价？——解析期新指令被丢；改进：双缓冲或 DMA 环形队列。

10.2 进阶改造路线

1. **上电自动陀螺零偏校准**：上电静置采 1s 平均 GY 替代硬编码 16；
2. **卡尔曼滤波**替代互补滤波（把 α 换成随角速度自适应的增益）；
3. **速度环前馈 + 低通**：编码器速度先过一阶低通再进 PID，抑制量化噪声；
4. **DMP 硬件解算**：MPU6050 内置 DMP 直接输出四元数，省掉 MCU 上的滤波计算；
5. **串口 DMA + 空闲中断**收包，替代逐字节中断，接收零 CPU 占用；
6. **看门狗**：IWDG 由控制中断喂狗，程序跑飞自动复位站立保护；
7. **参数掉电保存**：调好的 PID 存入 Flash/BKP 备份域，免每次手机重调。

附录 A 引脚分配总表

见 0.2 节表格（此处不重复）。补充电源要点：电机驱动必须**独立供电或大电容缓冲**，电机堵转电流会把 3.3V 拉垮导致 MCU 复位——表现为”一动就重启”。

附录 B 全局符号速查表

符号	类型	定义处	含义
Angle / AngleAcc / AngleGyro	float	main.c	融合角 / 加计角 / 陀螺角
AX..GZ	int16_t	main.c	MPU6050 六轴原始值
RunFlag / KeyNum	uint8_t	main.c	运行开关 / 键值
LeftPWM / RightPWM / AvePWM / DifPWM	int16_t	main.c	电机 PWM 及其分解
LeftSpeed / RightSpeed / AveSpeed / DifSpeed	float	main.c	轮速及分解
AnglePID / SpeedPID / TurnPID	PID_t	main.c	三个控制器
TimerErrorFlag / TimerCount	—	main.c	中断超时调试
OLED_DisplayBuf[8][128]	uint8_t	OLED.c	OLED 显存
Key_Num	uint8_t	Key.c	按键事件暂存
BlueSerial_RxPacket / RxFlag	—	BlueSerial.c	蓝牙包缓冲与就绪标志
Serial_RxData / RxFlag	—	Serial.c	调试串口单字节接收

> 全文完。建议配合源码对照阅读：先精读第 1 章 main.c 建立全景，再按”PID → 姿态 → 执行 → 节拍 → 交互 → 通信”的顺序逐个模块深入，最后回到第 9 章动手调参验证理解。